

Unit :2

Software Requirements Engineering And analysis Modeling

Content

- **Requirements Engineering:** User and system requirements, Functional and non-functional requirements, requirements engineering (elicitation, specification, validation, negotiation) prioritizing requirements (Kano diagram), requirement traceability matrix(RTM),
- **Software Requirements Specification (SRS):** software requirements Specification document, structure of SRS, writing a SRS, structured SRS for online shopping,
- **Requirements Analysis:** Analysis Model, data modeling, scenario based modeling, class based modeling, Flow oriented modeling, behavioral modeling-Introduction to UML diagrams

What is requirement?

- Types
 1. User requirement
 2. System requirement
 3. Software Specification

User requirements

- User requirements are statements, in a natural language plus diagrams, of what services the system is expected to provide to system users and the constraints under which it must operate.

System requirements

- System requirements are more detailed descriptions of the software system's functions, services, and operational constraints.
- The system requirements document (sometimes called a functional specification) should define exactly what is to be implemented.
- It may be part of the contract between the system buyer and the software developers.

Functional and non-functional requirements

- Functional requirements
 - These are statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations.
 - In some cases, the functional requirements may also explicitly state what the system should not do.

- Non-functional requirements
 - These are constraints on the services or functions offered by the system.
 - They include timing constraints, constraints on the development process, and constraints imposed by standards.
 - Non-functional requirements often apply to the system as a whole, rather than individual system features or services.

The Problems with our Requirements Practices

- We have trouble understanding the requirements that we do acquire from the customer
- We often record requirements in a disorganized manner
- We spend far too little time verifying what we do record
- We allow change to control us, rather than establishing mechanisms to control change
- Most importantly, we fail to establish a solid foundation for the system or software that the user wants built

(more on next slide)

The Problems with our Requirements Practices (continued)

- Many software developers argue that
 - Building software is so compelling that we want to jump right in (before having a clear understanding of what is needed)
 - Things will become clear as we build the software
 - Project stakeholders will be able to better understand what they need only after examining early iterations of the software
 - Things change so rapidly that requirements engineering is a waste of time
 - The bottom line is producing a working program and that all else is secondary
- All of these arguments contain some truth, especially for small projects that take less than one month to complete
- However, as software grows in size and complexity, these arguments begin to break down and can lead to a failed software project

A Solution: Requirements Engineering

- Begins during the communication activity and continues into the modeling activity
- Builds a bridge from the system requirements into software design and construction
- Allows the requirements engineer to examine
 - the context of the software work to be performed
 - the specific needs that design and construction must address
 - the priorities that guide the order in which work is to be completed
 - the information, function, and behavior that will have a profound impact on the resultant design

Requirements Engineering Tasks

- Seven distinct tasks
 - Inception
 - Elicitation
 - Elaboration
 - Negotiation
 - Specification
 - Validation
 - Requirements Management
- Some of these tasks may occur in parallel and all are adapted to the needs of the project
- All strive to define what the customer wants
- All serve to establish a solid foundation for the design and construction of the software

Inception

Elicitation

Elaboration

Negotiation

Specification
n

Validation

Requirements
Management

Inception Task

- During inception, the requirements engineer asks a set of questions to establish...
 - A basic understanding of the problem
 - The people who want a solution
 - The nature of the solution that is desired
 - The effectiveness of preliminary communication and collaboration between the customer and the developer
- Through these questions, the requirements engineer needs to...
 - Identify the stakeholders
 - Recognize multiple viewpoints
 - Work toward collaboration
 - Break the ice and initiate the communication

Elicitation Task

- Eliciting requirements is difficult because of
 - Problems of scope in identifying the boundaries of the system or specifying too much technical detail rather than overall system objectives
 - Problems of understanding what is wanted, what the problem domain is, and what the computing environment can handle (Information that is believed to be "obvious" is often omitted)
 - Problems of volatility because the requirements change over time
- Elicitation may be accomplished through two activities
 - Collaborative requirements gathering
 - Quality function deployment

Elicitation Work Products

The work products will vary depending on the system, but should include one or more of the following items

- A statement of need and feasibility
- A bounded statement of scope for the system or product
- A list of customers, users, and other stakeholders who participated in requirements elicitation
- A description of the system's technical environment
- A list of requirements (organized by function) and the domain constraints that apply to each
- A set of preliminary usage scenarios (in the form of use cases) that provide insight into the use of the system or product under different operating conditions
- Any prototypes developed to better define requirements

Elaboration Task

- During elaboration, the software engineer takes the information obtained during inception and elicitation and begins to expand and refine it
- Elaboration focuses on developing a refined technical model of software functions, features, and constraints
- It is an analysis modeling task
 - Use cases are developed
 - Domain classes are identified along with their attributes and relationships
 - State machine diagrams are used to capture the life on an object
- The end result is an analysis model that defines the functional, informational, and behavioral domains of the problem

Questions Commonly Answered by a Use Case

- Who is the primary actor(s), the secondary actor(s)?
- What are the actor's goals?
- What preconditions should exist before the scenario begins?
- What main tasks or functions are performed by the actor?
- What exceptions might be considered as the scenario is described?
- What variations in the actor's interaction are possible?
- What system information will the actor acquire, produce, or change?
- Will the actor have to inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

Negotiation Task

- During negotiation, the software engineer reconciles the conflicts between what the customer wants and what can be achieved given limited business resources
- Requirements are ranked (i.e., prioritized) by the customers, users, and other stakeholders
- Risks associated with each requirement are identified and analyzed
- Rough guesses of development effort are made and used to assess the impact of each requirement on project cost and delivery time
- Using an iterative approach, requirements are eliminated, combined and/or modified so that each party achieves some measure of satisfaction

The Art of Negotiation

- Recognize that it is not competition
- Map out a strategy
- Listen actively
- Focus on the other party's interests
- Don't let it get personal
- Be creative
- Be ready to commit

Specification Task

- A specification is the final work product produced by the requirements engineer
- It is normally in the form of a software requirements specification
- It serves as the foundation for subsequent software engineering activities
- It describes the function and performance of a computer-based system and the constraints that will govern its development
- It formalizes the informational, functional, and behavioral requirements of the proposed software in both a graphical and textual format

Validation Task

- During validation, the work products produced as a result of requirements engineering are assessed for quality
- The specification is examined to ensure that
 - all software requirements have been stated unambiguously
 - inconsistencies, omissions, and errors have been detected and corrected
 - the work products conform to the standards established for the process, the project, and the product
- The formal technical review serves as the primary requirements validation mechanism
 - Members include software engineers, customers, users, and other stakeholders

Questions to ask when Validating Requirements

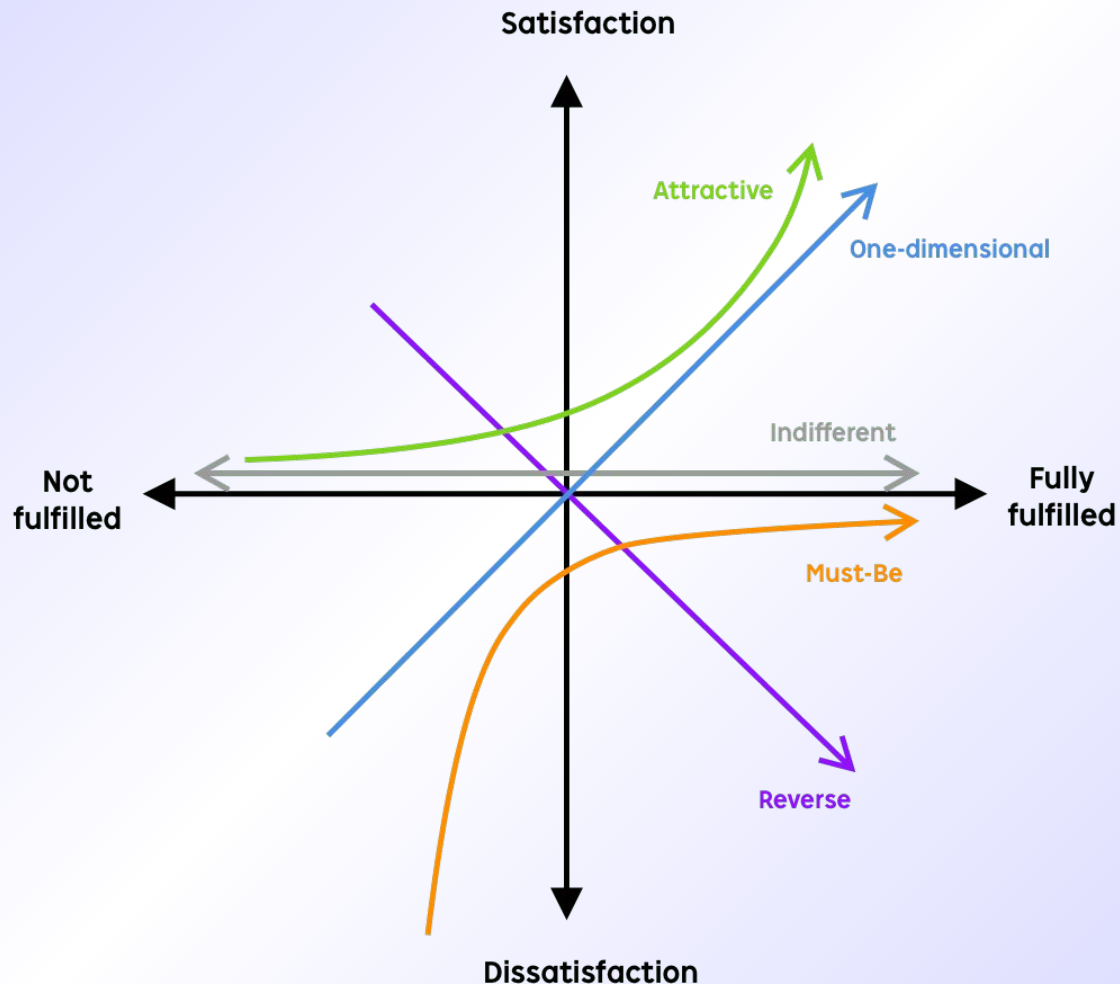
- Is each requirement consistent with the overall objective for the system/product?
- Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
- Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?
- Is each requirement bounded and unambiguous?
- Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?

(more on next slide)

Requirements Management Task

- During requirements management, the project team performs a set of activities to identify, control, and track requirements and changes to the requirements at any time as the project proceeds
- Each requirement is assigned a unique identifier
- The requirements are then placed into one or more traceability tables
- These tables may be stored in a database that relate features, sources, dependencies, subsystems, and interfaces to the requirements
- A requirements traceability table is also placed at the end of the software requirements specification

Prioritizing Requirements (Kano diagram)



What is the Kano Model?

- The Kano Model is a framework developed by Professor Noriaki Kano to prioritize and analyze customer needs and preferences.
- It was introduced in the 1980s and has been widely used in product development and customer satisfaction research.
- The model categorizes features or attributes of a product into five distinct categories based on how they impact customer satisfaction.

- Kano Model is a method for ranking features on a product roadmap according to how probable they are to satisfy customers.
- Product teams can assess if it makes strategic sense to add a high-satisfaction feature to the roadmap by comparing its implementation costs to those of the feature.

Categories of Kano Model

- Basic Needs (Must-Have)
- Performance Needs (More is Better)
- Excitement Needs (Delighters)
- Indifferent Needs (Take It or Leave It)
- Reverse Needs (Dissatisfaction When Present)

Basic Needs (Must-Have)

- Basic needs are essential functions that clients anticipate as a minimum requirement.
- These are the vital, non-negotiable features or attributes of a services or products.
- When those functions are present, clients do not longer grow to be pretty happy, however their absence can cause dissatisfaction.

Performance Needs (More is Better)

- Performance needs are functions which have a linear relationship with patron pleasure.
- This method that as you improve these capabilities, customer pride increases proportionally.
- However, the absence of these features does now not always lead to dissatisfaction.

Excitement Needs (Delighters)

- Excitement needs are sudden or delightful functions which have the ability to seriously beautify purchaser satisfaction.
- They are the features that set a product aside from its competition.

Indifferent Needs (Take It or Leave It)

- Indifferent desires are capabilities that don't substantially effect patron pride.
- Whether these functions are gift or absent, client pleasure remains pretty constant.

Reverse Needs (Dissatisfaction When Present)

- Reverse desires are functions that, when gift, can in reality cause dissatisfaction.
- However, when they may be absent, they do not necessarily increase consumer satisfaction.

When Should You Use the Kano Model?

- **Limited Time:** When we have limited time to build any product, Kano model is fruitful in that case.
- **Limited Resources:** When we have limited time to build a product, There is no need to worry about time and implement the Kano model.
- **Product Improvement:** For present products or services, it aids in identifying regions for enhancement.
- **Competitive Analysis:** It can reveal how your product compares to competition in terms of consumer delight.
- **Customer Feedback Analysis:** Use the Kano Model to categorize and prioritize patron comments and suggestions.

Requirement Traceability Matrix(RTM)

- RTM maps all the requirements with the test cases.
- By using this document one can verify test cases cover all functionality of the application as per the requirements of the customer.
- **Requirements:** Requirements of a particular project from the client.
- **Traceability:** The ability to trace the tests.
- **Matrix:** The data which can be stored in rows and columns form.

- Is a document that maps and traces user requirement with test cases.
- It captures all requirements proposed by the client and requirement traceability in a single document, delivered at the conclusion of the Software development life cycle.
- The main purpose of Requirement Traceability Matrix is to validate that all requirements are checked via test cases such that no functionality is unchecked during Software testing.

Requirement Traceability Matrix

	Test Case ID →	TC_1	TC_2	TC_3	TC_4	TC_5	TC_6	TC_7	TC_8	TC_9	TC_10	# Test Cases for respective Requirement
Req. ID ↓												
Req_1		✗		✗			✗					3
Req_2			✗			✗						2
Req_3				✗								1
Req_4					✗		✗					2
Req_5						✗		✗				2
Req_6							✗					1
Req_7						✗		✗				2
Req_8									✗			1
Req_9										✗		1
Req_10											✗	1

© TestingClub.blogspot

Software Requirements Specification (SRS)

- Software Requirement Specification (SRS) Format as the name suggests, is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system.
- These requirements can be functional as well as non-functional depending upon the type of requirement.
- The interaction between different customers and contractors is done because it is necessary to fully understand the needs of customers.

Structure of SRS

1. Purpose

- i. Definitions
- ii. Background
- iii. System overview
- iv. References

2. Overall description

- i. Product perspective
 - a) System Interfaces
 - b) User interfaces
 - c) Hardware interfaces
 - d) Software interfaces
 - e) Communication Interfaces
 - f) Memory constraints
- ii. Design constraints
 - a) Operations
 - b) Site adaptation requirements
- iii. Product functions
- iv. User characteristics
- v. Constraints, assumptions and dependencies

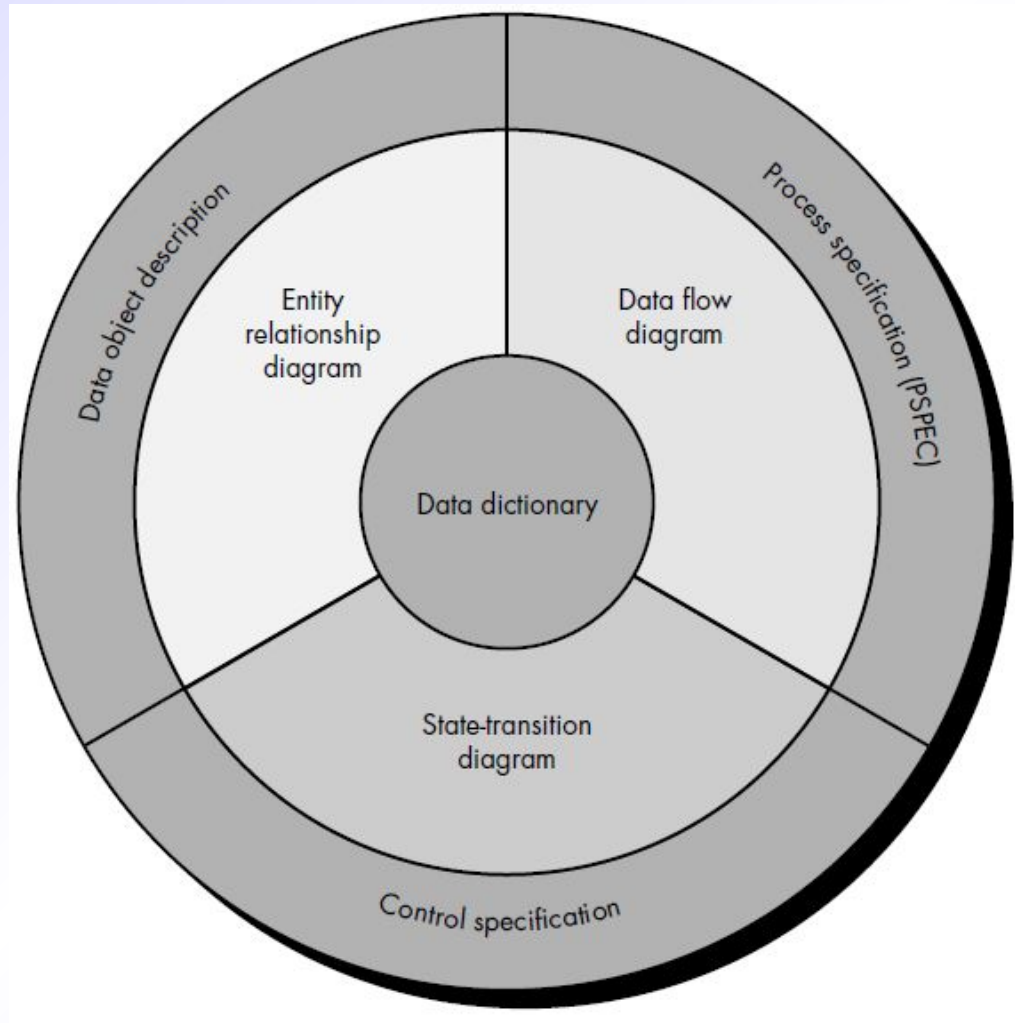
4. Specific requirements

- i. External interface requirements
- ii. Performance requirements
- iii. Logical database requirement
- iv. Software system attributes
 - a) Reliability
 - b) Availability
 - c) Security
 - d) Maintainability
 - e) Portability
- v. Functional requirements
 - a) Functional partitioning
 - b) Functional description
 - c) Control description
- vi. Environment characteristics
 - a) Hardware
 - b) Peripherals
 - c) Users
- vii. Other

Elements of the Analysis Model

- Scenario-based elements
 - Describe the system from the user's point of view using scenarios that are depicted in use cases and activity diagrams
- Class-based elements
 - Identify the domain classes for the objects manipulated by the actors, the attributes of these classes, and how they interact with one another; they utilize class diagrams to do this
- Behavioral elements
 - Use state diagrams to represent the state of the system, the events that cause the system to change state, and the actions that are taken as a result of a particular event; can also be applied to each class in the system
- Flow-oriented elements
 - Use data flow diagrams to show the input data that comes into a system, what functions are applied to that data to do transformations, and what resulting output data are produced

The structure of the analysis model



- At the core of the model lies the **data dictionary**—a repository that contains descriptions of all data objects consumed or produced by the software.
- Three different diagrams surround the core.
- The **entity relation diagram (ERD)** depicts relationships between data objects.
- The ERD is the notation that is used to conduct the data modeling activity.
- The attributes of each data object noted in the ERD can be described using a data object description.

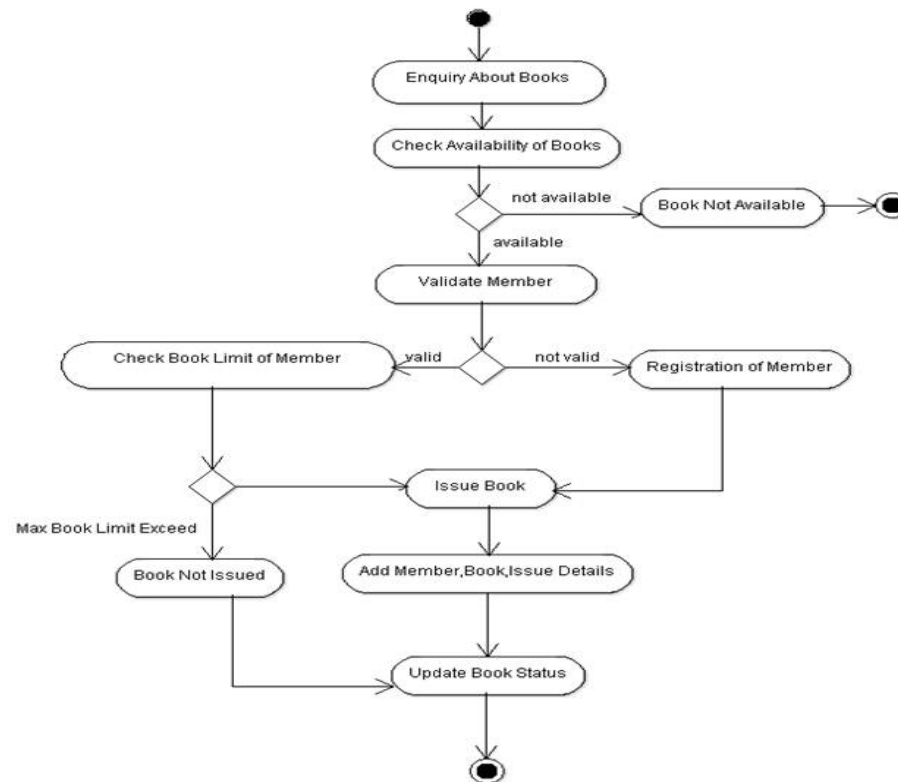
- The data flow diagram (DFD) serves two purposes:
 - to provide an indication of how data are transformed as they move through the system
 - to depict the functions (and subfunctions) that transform the data flow.
- A description of each function presented in the DFD is contained in a process specification (PSPEC).

- The state transition diagram (STD) indicates how the system behaves as a consequence of external events.
- To accomplish this, the STD represents the various modes of behavior (called states) of the system and the manner in which transitions are made from state to state.
- The STD serves as the basis for behavioral modeling.
- Additional information about the control aspects of the software is contained in the control specification (CSPEC).

DATA MODELING

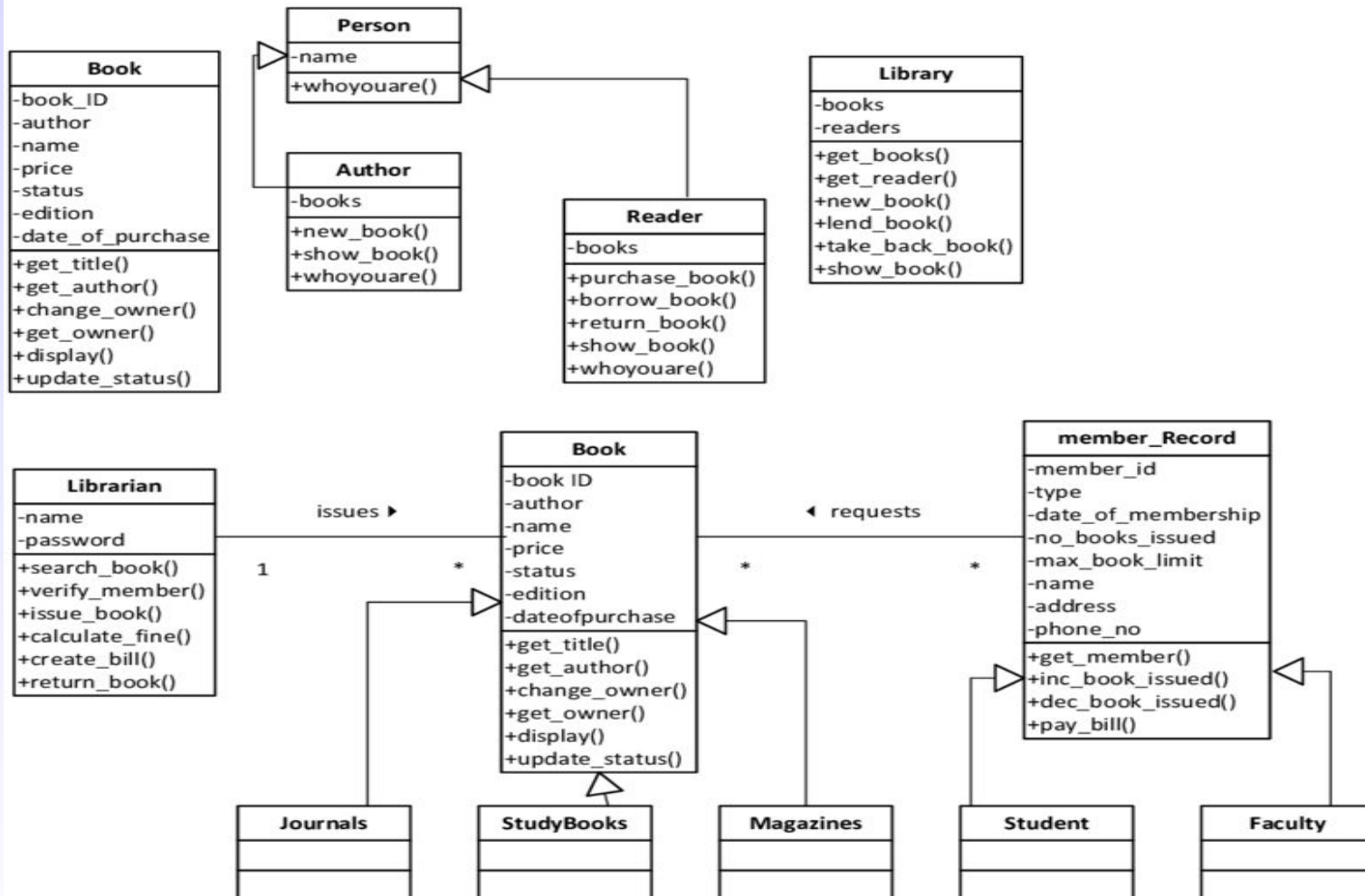
- Data modeling answers a set of specific questions that are relevant to any data processing application.
 - What are the primary data objects to be processed by the system?
 - What is the composition of each data object and what attributes describe the object?
 - Where do the objects currently reside? What are the relationships between each object and other objects?
 - What are the relationships between the objects and the processes that transform them?

Scenario Based element

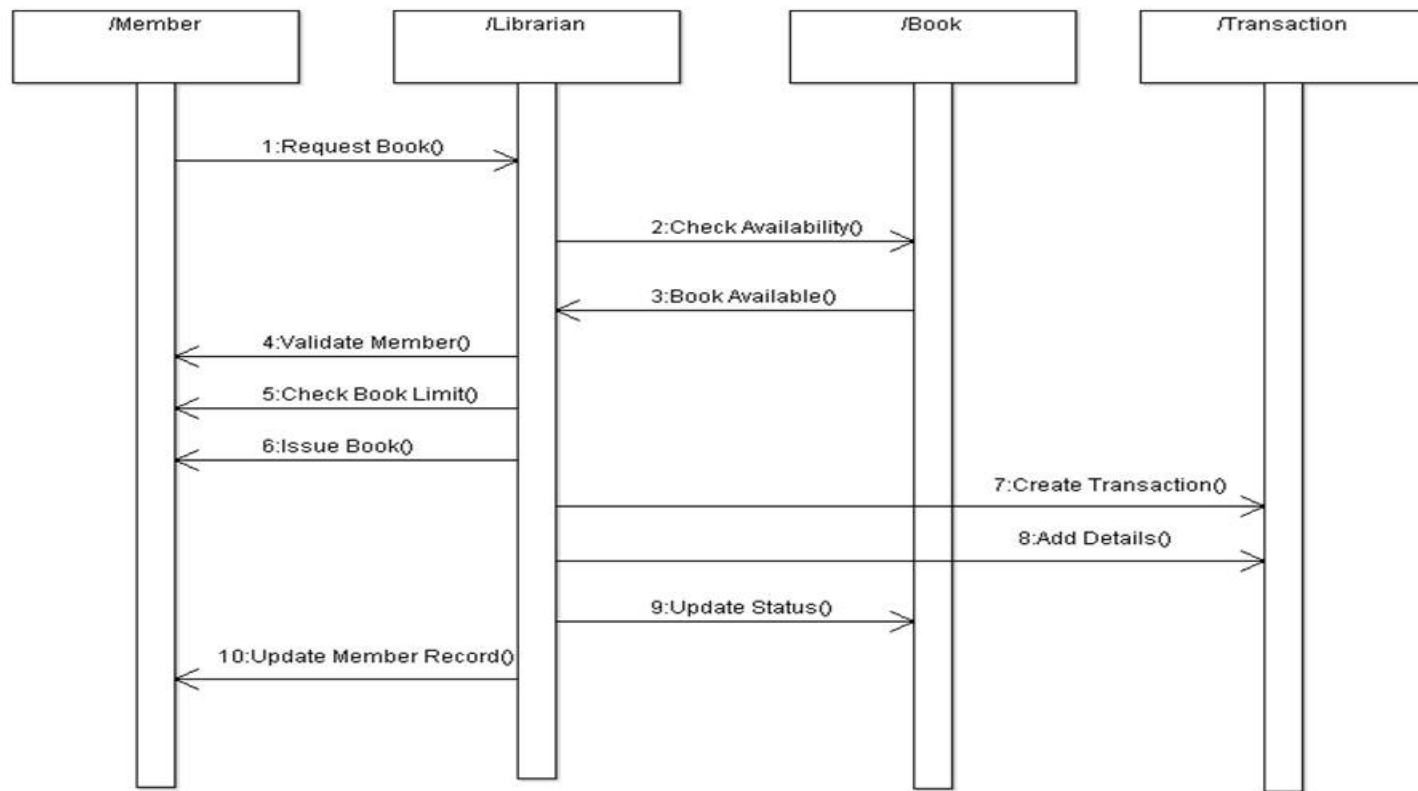


dig.Activity Digram for Issuing Book from Library

Class based elements



Behavioral elements



dig.Sequence Diagram For Issuing Book From Library

Flow Model

